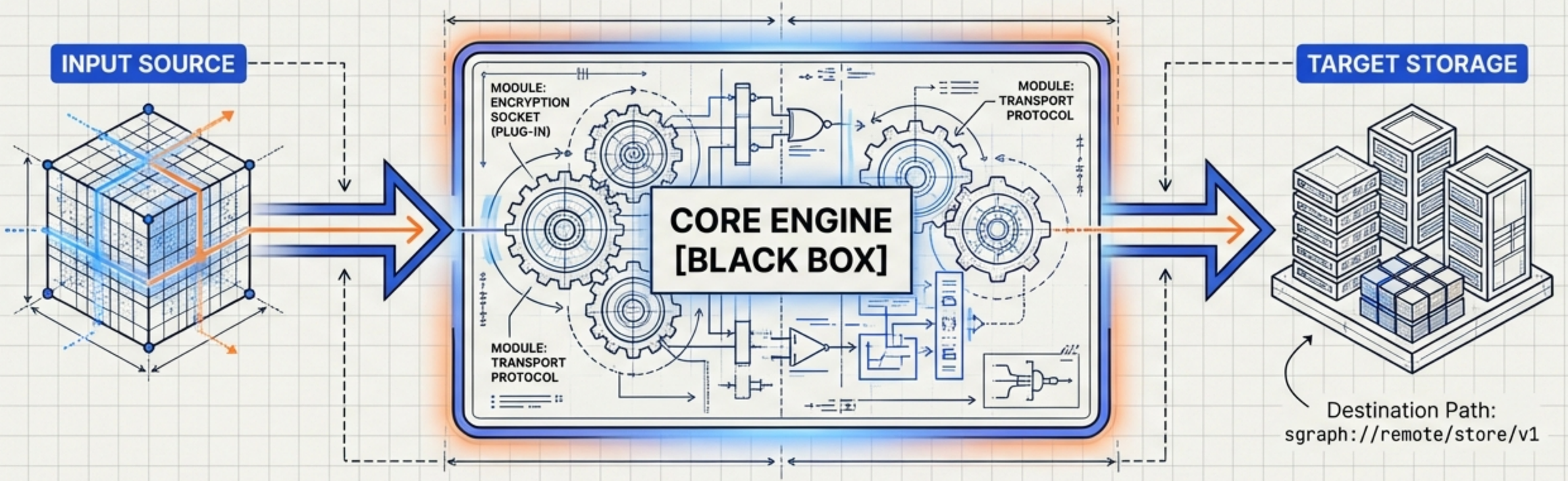
THE FILE TRANSFER ENGINE: ARCHITECTURE & RESEARCH BRIEF

v0.3.12 | Building the Core of SGraph Send

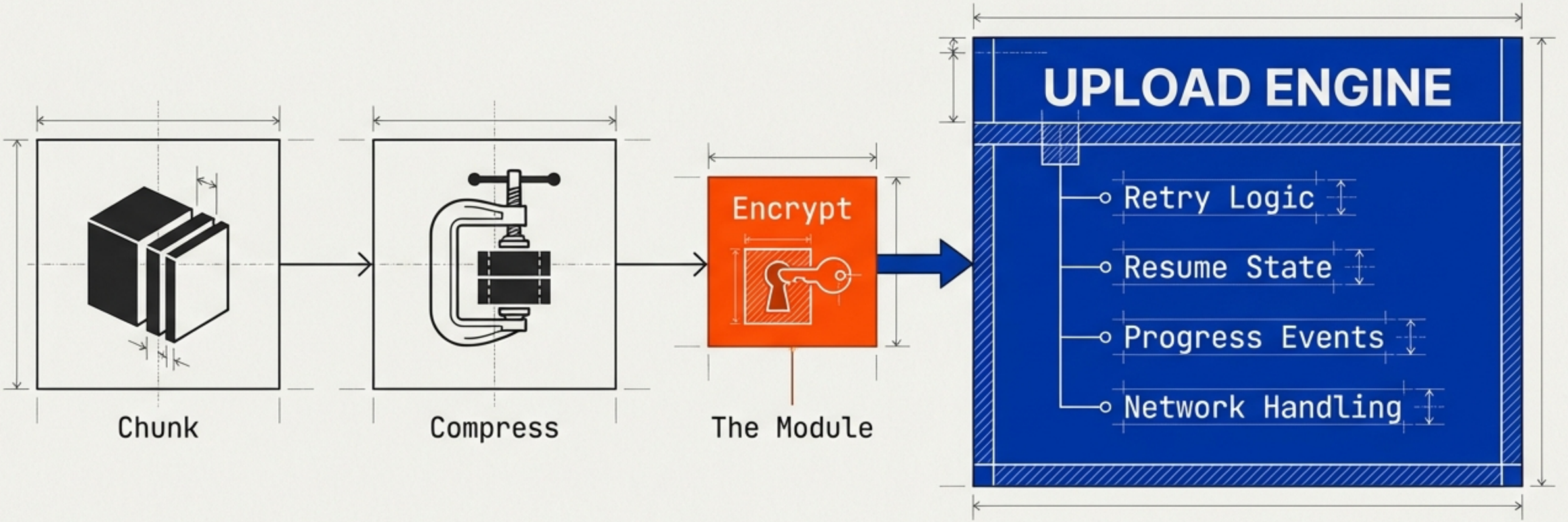
- FROM: Human (Project Lead)
- TO: Explorer Team (Architect, Dev, AppSec, DevOps)
- DATE: 15 Feb 2026
- STATUS: Exploration Workstreams Triggered



CORE THESIS: This component is fundamental enough to deserve its own project. **We are building the transport engine first; encryption is a module that plugs into the pipeline.**

Encryption is Just a Callback

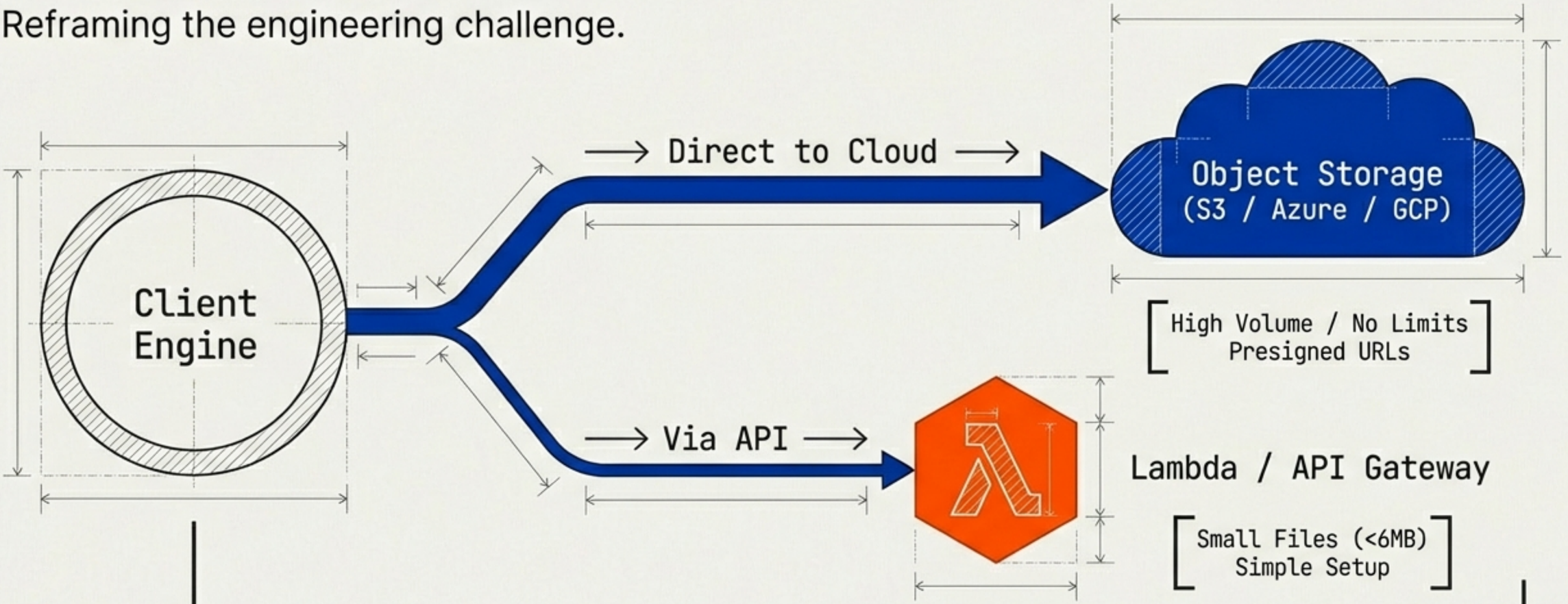
Reframing the engineering challenge.



Key Insight: The real challenge isn't cryptography; it is getting bits from A to B reliably at scale. If the transfer fails, the best crypto in the world won't save the User Experience.

Transport Agnostic Primitives

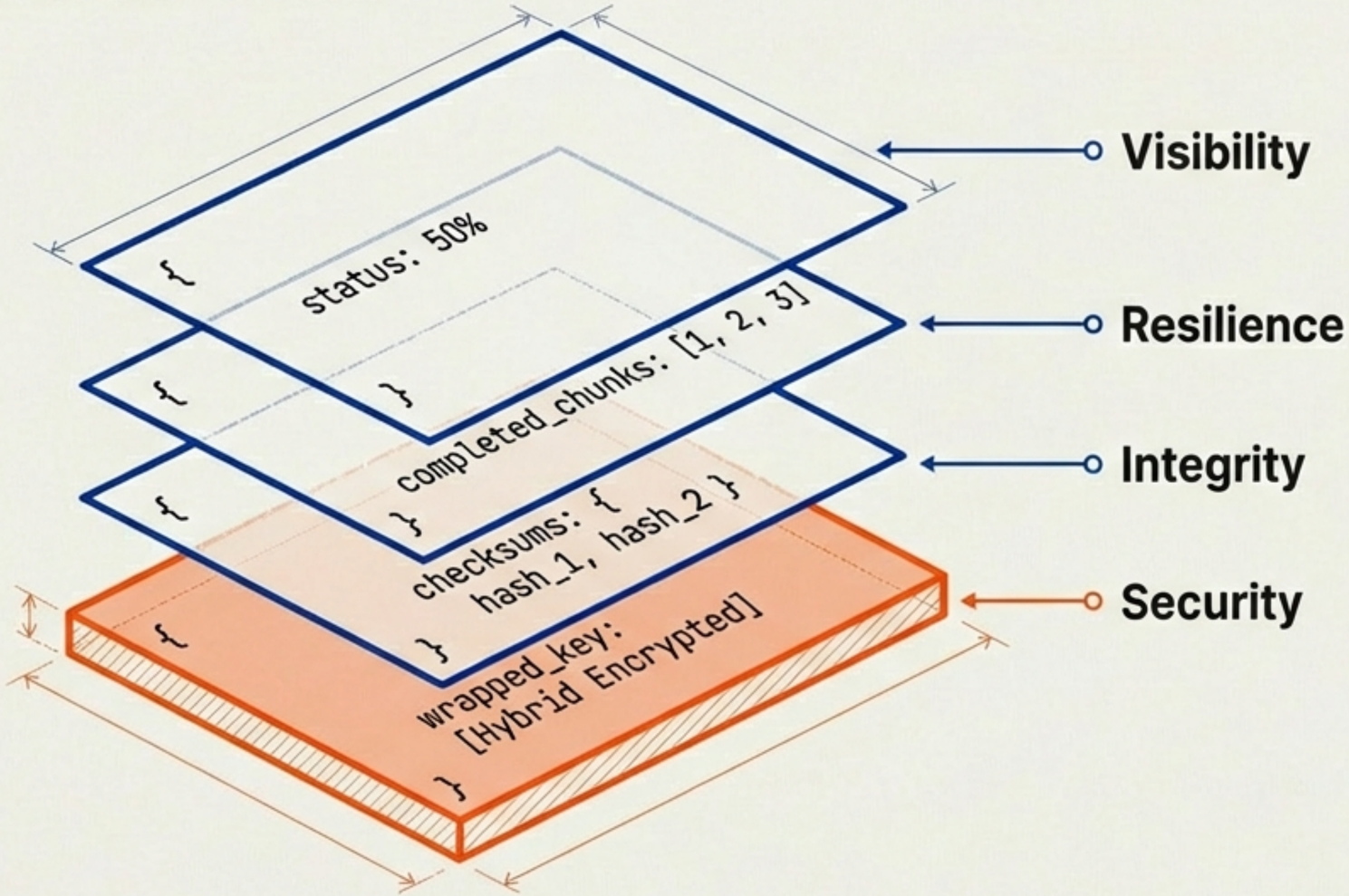
Reframing the engineering challenge.



The Unifying Logic: Chunking, progress, and resilience logic remains identical regardless of the transport path.

The Brain: The Transfer Manifest

Single Source of Truth



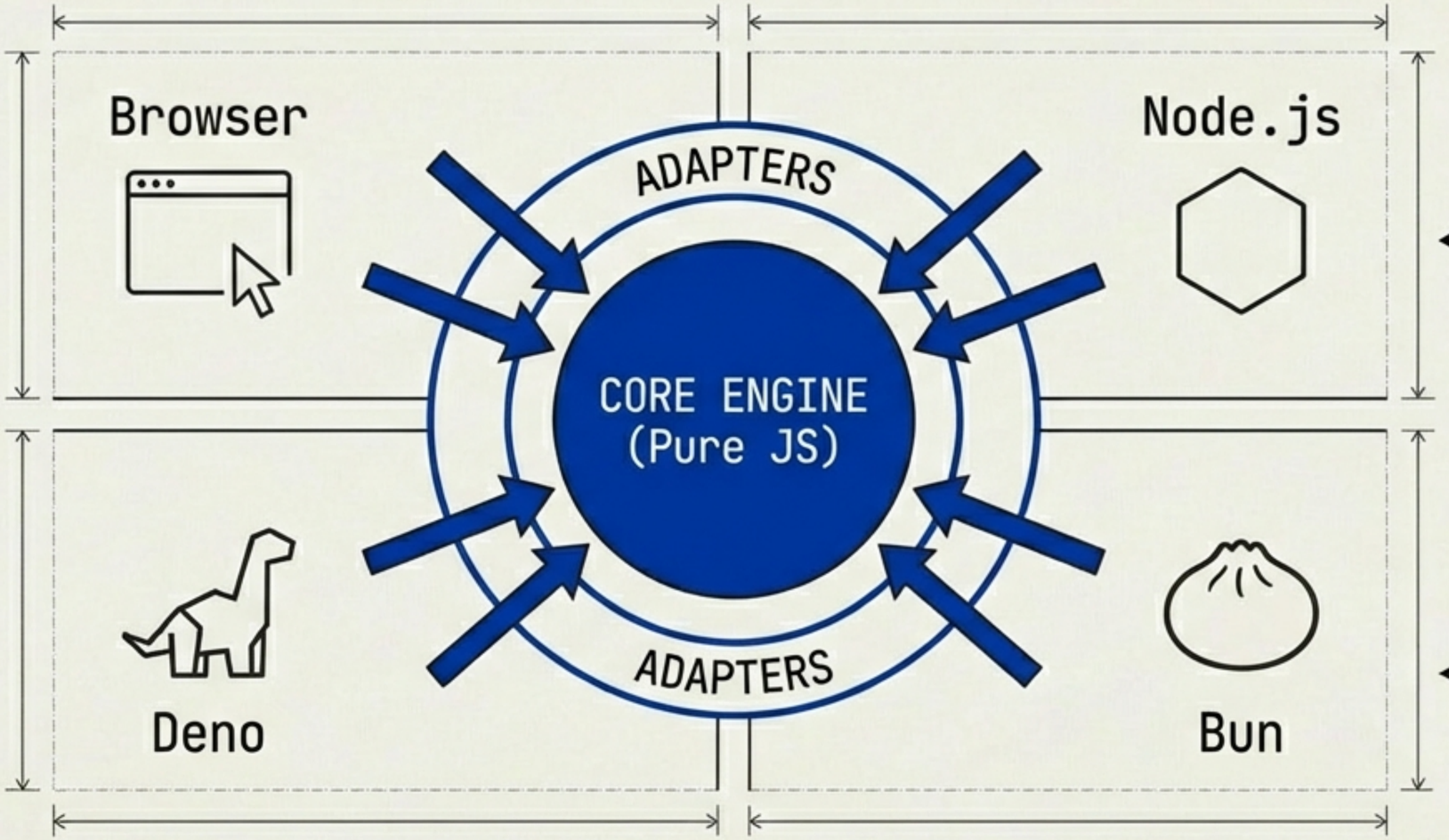
Hybrid Encryption Protocol:

1. Generate random AES-256 key per file.
2. Encrypt data with AES key.
3. Wrap AES key with User Passphrase.

Result: Plaintext key is never stored.

Multi-Runtime from Day One

Reframing the engineering challenge.

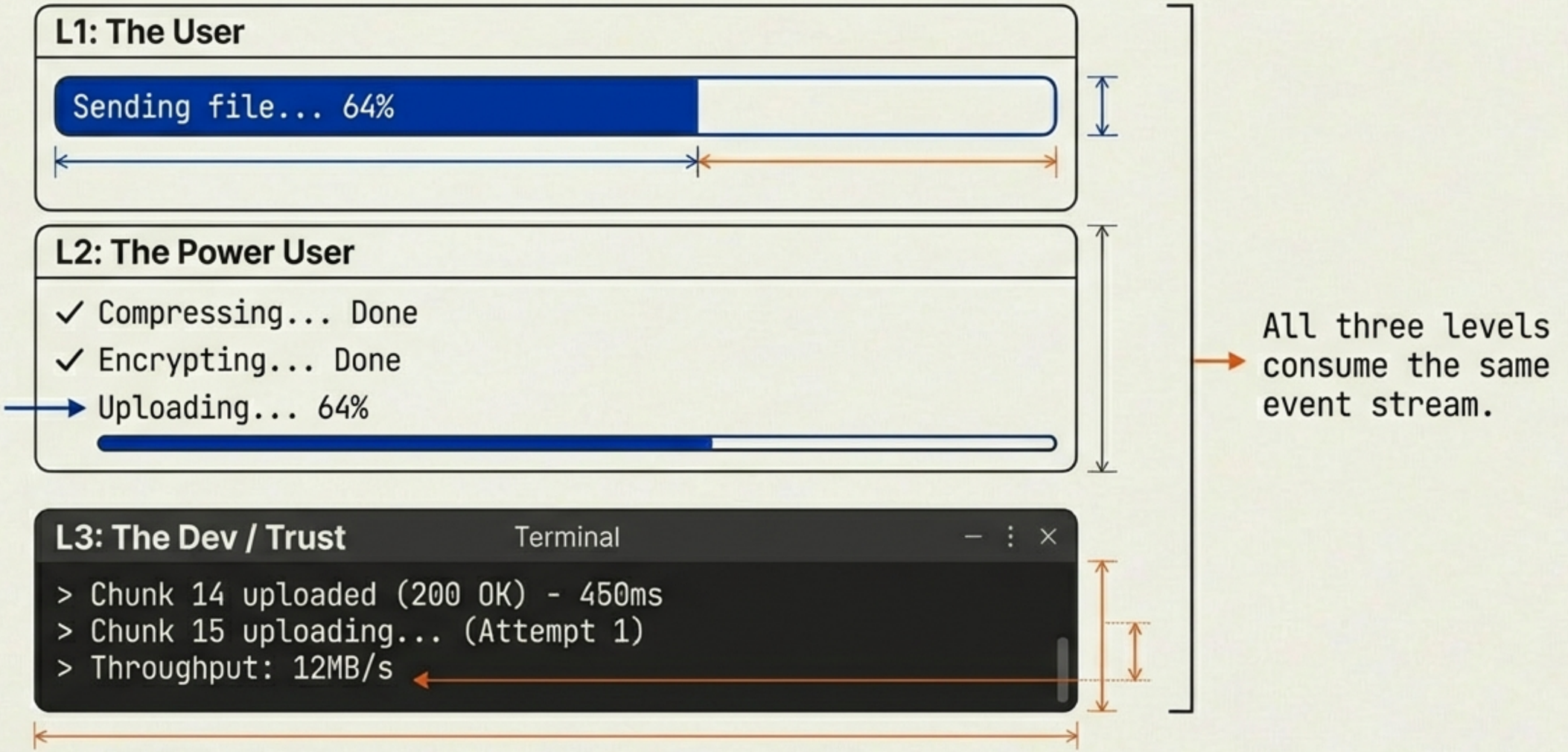


Strategic Advantage:

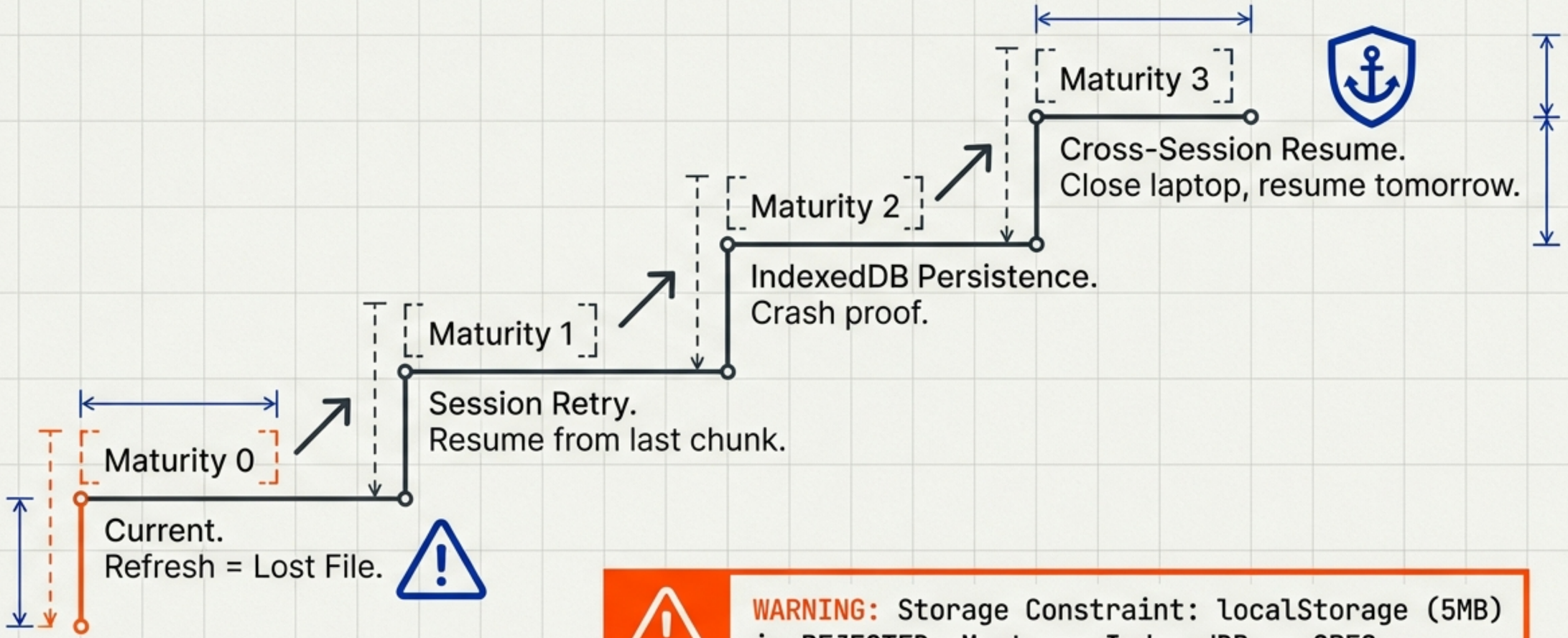
We stress-test the engine via CLI/CI without ever opening a browser. By the time code reaches the UI, it is already proven.

The Unifying Logic: Chunking, progress, and resilience logic remains identical regardless of the transport path.

Principle 1: Maximum Visibility



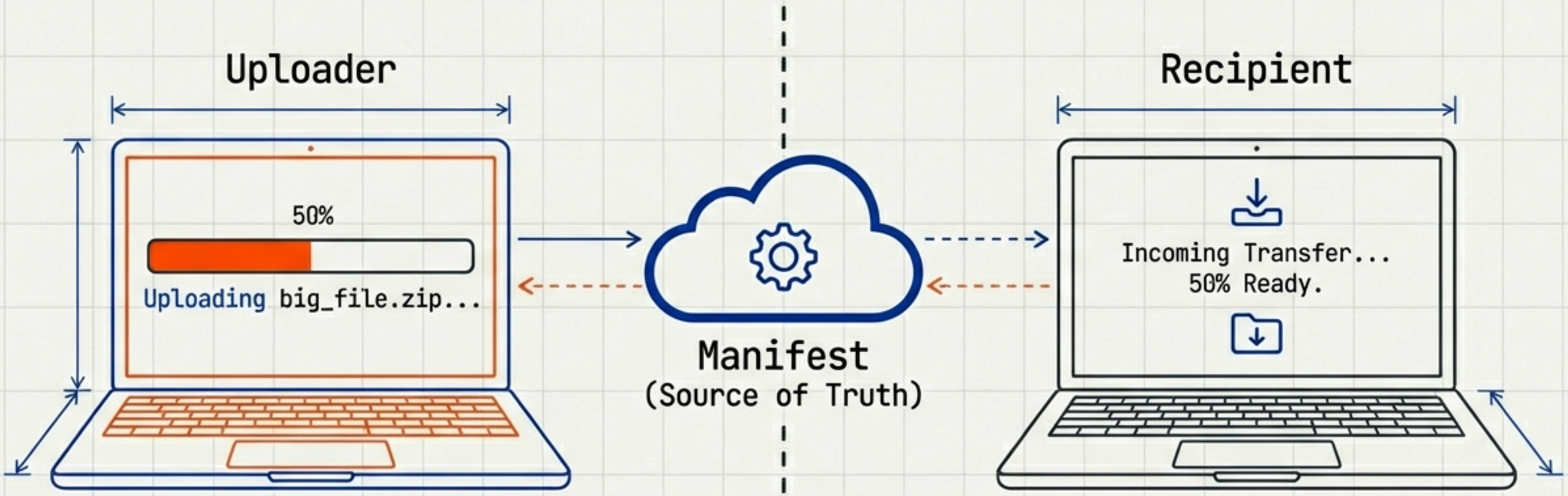
Principle 2: Zero Data Loss



WARNING: Storage Constraint: localStorage (5MB) is REJECTED. Must use IndexedDB or OPFS.

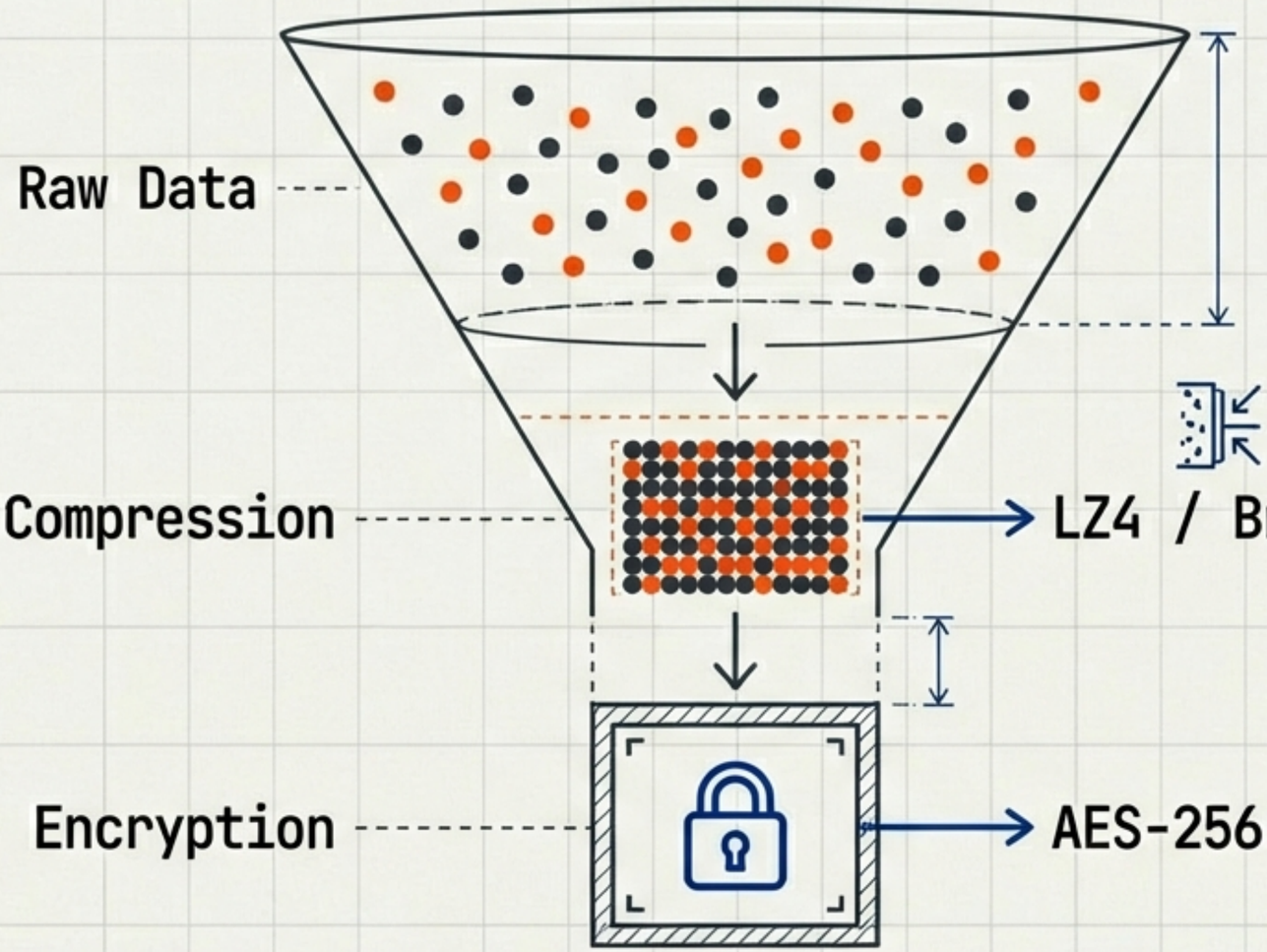
Principle 3: Reducing User Anxiety

The "Is It There Yet?" Problem



The Manifest allows the Downloader to see the Uploader’s status in real-time. Turns a stressful refresh-cycle into a calm wait.

Efficiency & Resource Management



THE GOLDEN RULE:

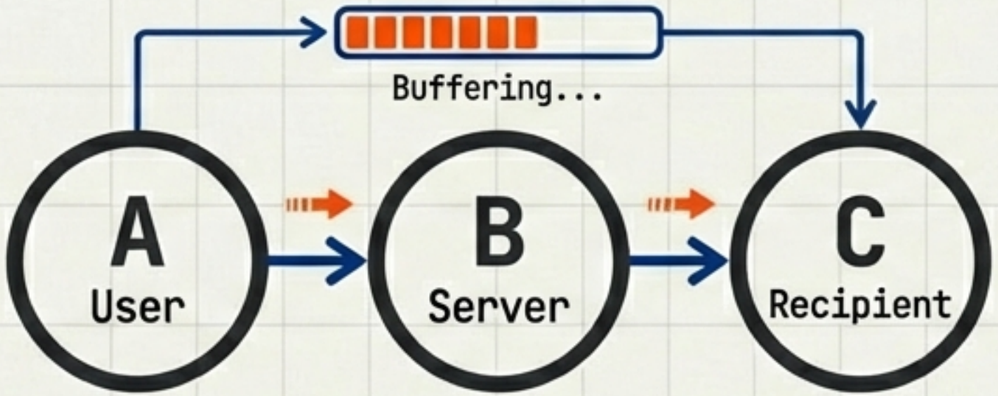
Never encrypt first. Encrypted data is random and cannot be compressed.

The order must be Compress -> Encrypt -> Upload.

Advanced Transport Modes

A

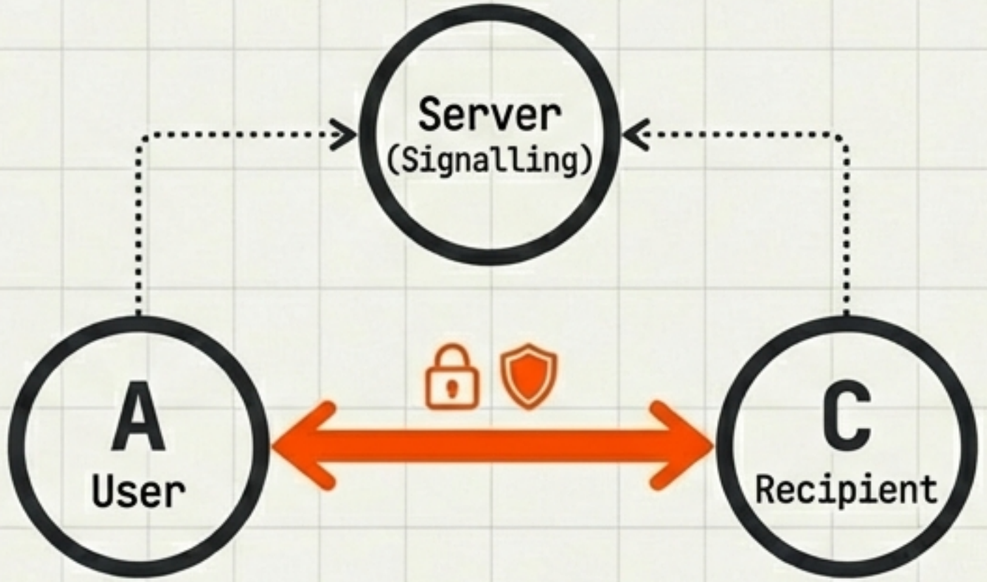
Mode: Relay / Streaming



Buffers chunks. Download begins before upload ends.

B

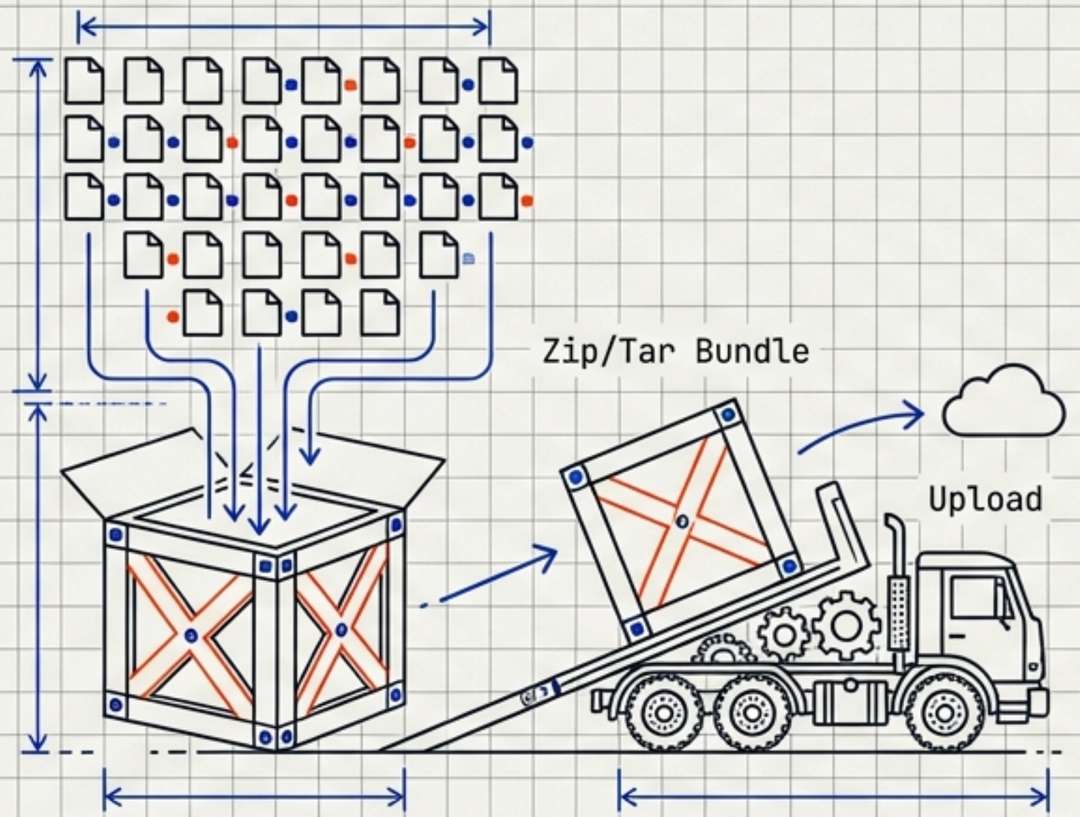
Mode: WebRTC (Research)



Peer-to-Peer. Zero server bandwidth. Maximum privacy.

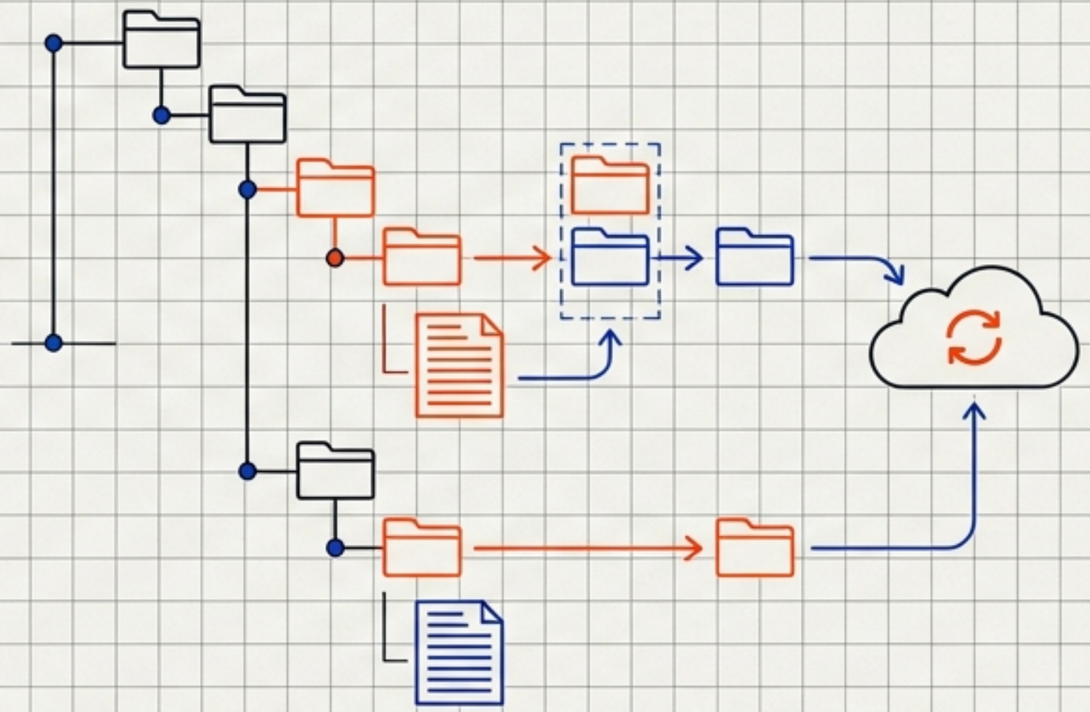
Roadmap: Bundling & Folder Sync

Bundling Strategy



Reduces S3 request overhead.
1000 files -> 1 Bundle

Folder Sync (Future)



Delta Sync.
Only modified chunks are uploaded.
'Git for file sharing'.

Research: Don’t Reinvent the Wheel

Investigation Checklist

Platform Natives (S3/Azure)	Open Protocols	Custom Build
- Multipart Upload ✓ - Transfer Acceleration ✓ - Presigned URLs ✓ - Resumable Uploads ✓	- tus.io (Resumable Protocol) - Uppy Ecosystem ✓	- Only for Encryption / Manifest logic ✓ - WASM (Backup only) ✓

Goal: Build ON TOP of the best infrastructure primitives.

Implementation Roadmap

ACTIVE PHASE | FUTURE PHASE | FUTURE PHASE | FUTURE PHASE

Core & CLI

Q1

- Adapter Interfaces
- Core Engine Logic
- CLI Tool for Testing

Browser Integration

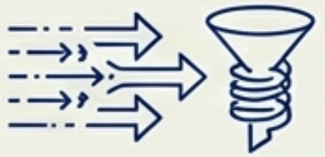
Q2

- Web Crypto Adapter
- IndexedDB Cache
- 3-Level UI

Advanced

Q3

- Streaming Mode
- Compression Pipeline



Scale

Q4

- Deduplication
- WASM Optimisation

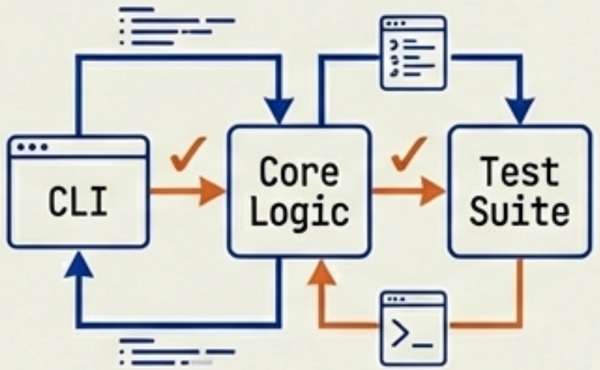
ACTIVE PHASE | FUTURE PHASE | FUTURE PHASE | FUTURE PHASE

TECHNICAL DATE

The Path Forward

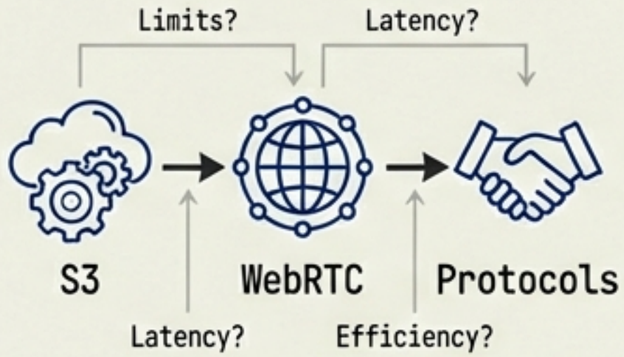
1) Prioritise Phase 1

Build Core + CLI + Tests first. Validate logic before UI.



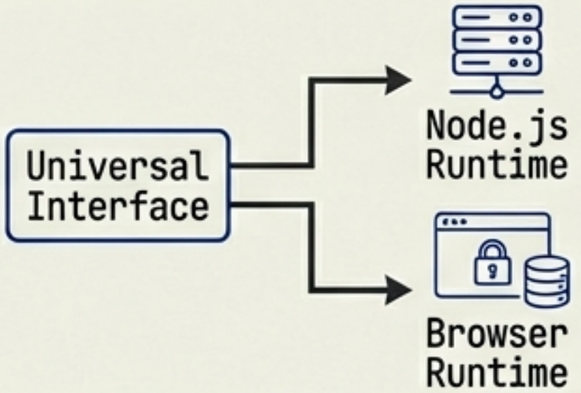
2) Launch Research

Investigate S3 limits, WebRTC, and Protocol evaluation immediately.



3) Design Adapters

Ensure interfaces support Multi-Runtime (Node + Browser) from day one.



Build the transfer engine right, and encryption is trivial to add. Build it wrong, and no amount of clever cryptography will save the user experience.